Tarea 53

Nombre: Yamil Salomón Saldaña

Módulo: Modelos de regresión lineal y series de tiempo

Actividad: Proyecto de Análisis de Regresión y Validación de Supuestos.

Desarrollo del Modelo e Intervalo de Confianza

Para este análisis se utilizó el dataset "Advertising.csv". Se realizó una partición de datos del 70% para entrenamiento y 30% para prueba con una semilla de aleatoriedad igual a 1. Tras un análisis inicial, se descartó la variable "Newspaper" por no ser estadísticamente significativa (P-value > 0.05). El modelo final utiliza únicamente TV y Radio como predictores, logrando un ajuste excelente con un R-cuadrado de 0.895.

```python
import pandas as pd
import numpy as np

# 1. Ingesta de datos
ruta = r"C:\Users\Yamil\OneDrive\Escritorio\Escritorio\Yamil\EBAC (CIENCIA DE DATOS)\Bloque 9 Análisis de regresión\MOD 3 Modelos RLineal\Practica\Cienti
df = pd.read_csv(ruta)

# 2. Inspección inicial de la estructura y limpieza
df.info()
df.head()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   TV         200 non-null    float64
 1   Radio      200 non-null    float64
 2   Newspaper  200 non-null    float64
 3   Sales      200 non-null    float64
dtypes: float64(4)
memory usage: 6.4 KB
```

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9

```python
from sklearn.model_selection import train_test_split
import statsmodels.api as sm

# 3. Separamos variables explicativas (X) y la dependiente (y)
X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']

# 4. Partición 70/30 con semilla en 1 (como pide la rúbrica)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=1)

# 5. Modelo base para revisar la significancia de las variables
X_train_const = sm.add_constant(X_train)
modelo_base = sm.OLS(y_train, X_train_const).fit()

# Imprimimos el resumen para auditar los P-values
print(modelo_base.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Sales   R-squared:                       0.899
Model:                            OLS   Adj. R-squared:                  0.897
Method:                 Least Squares   F-statistic:                     405.2
Date:                Wed, 15 Apr 2026   Prob (F-statistic):           1.36e-67
Time:                        17:58:43   Log-Likelihood:                -272.35
No. Observations:                 140   AIC:                             552.7
Df Residuals:                     136   BIC:                             564.5
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          4.6614      0.368     12.650      0.000       3.933       5.390
TV             0.0550      0.002     32.747      0.000       0.052       0.058
Radio          0.1025      0.010      9.801      0.000       0.082       0.123
Newspaper     -0.0015      0.007     -0.203      0.839      -0.016       0.013
==============================================================================
Omnibus:                       15.370   Durbin-Watson:                   2.041
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               25.876
Skew:                          -0.526   Prob(JB):                     2.41e-06
Kurtosis:                       4.824   Cond. No.                         432.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

Se calculó un intervalo de confianza al 90% para un escenario de inversión de TV=100 y Radio=50. Los resultados muestran que, bajo estas condiciones, el pronóstico puntual es de 16.04 unidades de venta.

```python
# 6. Reconstruimos el modelo eliminando la variable no significativa (Newspaper)
X_train_final = X_train[['TV', 'Radio']]

# 7. Ajustamos el modelo definitivo
X_train_final_const = sm.add_constant(X_train_final)
modelo_final = sm.OLS(y_train, X_train_final_const).fit()

# 8. Pronóstico de intervalo (90% de confianza) para TV=100, Radio=50
# Nota: Omitimos Newspaper (70) porque demostramos que no es estadísticamente significativa
nueva_inversion = [1, 100, 50] # [Constante, TV, Radio]
prediccion = modelo_final.get_prediction(nueva_inversion)
resumen_pred = prediccion.summary_frame(alpha=0.10) # 90% de confianza -> alpha = 0.10

print("--- Resumen del Modelo Final ---")
print(f"R-cuadrado: {modelo_final.rsquared:.4f}")
print("\n--- Pronóstico de Ventas (90% de Confianza) ---")
print(resumen_pred[['mean', 'obs_ci_lower', 'obs_ci_upper']])
```

```
--- Resumen del Modelo Final ---
R-cuadrado: 0.8993

--- Pronóstico de Ventas (90% de Confianza) ---
        mean  obs_ci_lower  obs_ci_upper
0  15.221676     12.342813     18.100538
```

Validación de Supuestos (Auditoría Técnica)

A. Supuesto de Normalidad (Jarque-Bera)

Se evaluó la normalidad de los residuales mediante la prueba de Jarque-Bera a un 90% de confianza. Se verificó el cálculo manual del sesgo y la curtosis contra las librerías de Python, obteniendo un empate exacto.

Interpretación: El estadístico JB (24.66) es mayor al valor crítico (4.60), por lo que se rechaza la normalidad de los residuales.

```python
import scipy.stats as stats

# 9. Extraemos los residuales de nuestro modelo final
y_pred = modelo_final.predict(X_train_final_const)
residuales = y_train - y_pred
n = len(residuales)
media_res = residuales.mean()

# 10. Cálculo Manual de Sesgo y Curtosis (Fórmulas de la lección)
m2 = sum((residuales - media_res)**2) / n
m3 = sum((residuales - media_res)**3) / n
m4 = sum((residuales - media_res)**4) / n

sesgo_manual = m3 / (m2**(3/2))
curtosis_manual = m4 / (m2**2)
jb_manual = (n / 6) * (sesgo_manual**2 + ((curtosis_manual - 3)**2) / 4)

# 11. Cálculo con Librerías (scipy.stats)
sesgo_lib = stats.skew(residuales, bias=True)
curtosis_lib = stats.kurtosis(residuales, fisher=False)

# 12. Obtención del valor crítico (Nivel de confianza 90% -> alpha 0.10)
chi2_critico = stats.chi2.ppf(0.90, df=2)

print("--- Comparativa Manual vs Librería ---")
print(f"Sesgo    -> Manual: {sesgo_manual:.4f} | Librería: {sesgo_lib:.4f}")
print(f"Curtosis -> Manual: {curtosis_manual:.4f} | Librería: {curtosis_lib:.4f}")
print(f"\nEstadístico Jarque-Bera (JB): {jb_manual:.4f}")
print(f"Valor Crítico (Chi-cuadrada al 90%): {chi2_critico:.4f}")
```

```
--- Comparativa Manual vs Librería ---
Sesgo    -> Manual: -0.5124 | Librería: -0.5124
Curtosis -> Manual: 4.7826 | Librería: 4.7826

Estadístico Jarque-Bera (JB): 24.6629
Valor Crítico (Chi-cuadrada al 90%): 4.6052
```

B. Supuesto de Autocorrelación (Durbin-Watson)

Texto: Se validó la inexistencia de autocorrelación. El cálculo manual coincide con el de la librería statsmodels.

- **Interpretación:** El valor de 2.0375 es aproximadamente 2, lo que indica ausencia de autocorrelación entre los términos de error.

```python
# 13. Cálculo Manual de Durbin-Watson (Fórmula de la lección)
# Durbin-Watson = Suma de las diferencias al cuadrado / Suma de los residuales al cuadrado
dif_residuales = residuales.diff().dropna() # e_t - e_{t-1}
suma_dif_cuad = sum(dif_residuales**2)
suma_res_cuad = sum(residuales**2)
dw_manual = suma_dif_cuad / suma_res_cuad

# 14. Cálculo con Librería (statsmodels)
from statsmodels.stats.stattools import durbin_watson
dw_lib = durbin_watson(residuales)

print("--- Supuesto 2: Autocorrelación (Durbin-Watson) ---")
print(f"Durbin-Watson -> Manual: {dw_manual:.4f} | Librería: {dw_lib:.4f}")
```

```
--- Supuesto 2: Autocorrelación (Durbin-Watson) ---
Durbin-Watson -> Manual: 2.0375 | Librería: 2.0375
```

C. Supuesto de Homocedasticidad (Prueba de White)

Se realizó la prueba de White para verificar la igualdad de varianzas en los residuales. El estadístico se calculó manualmente mediante una regresión auxiliar.

- **Interpretación:** El estadístico (11.35) supera el valor crítico (9.23), indicando evidencia de heterocedasticidad a un nivel de confianza del 90%.

```python
from statsmodels.stats.diagnostic import het_white

# 15. Preparación de datos para la regresión auxiliar de White
resid_sq = residuales ** 2
df_aux = X_train_final.copy()
# Creamos las variables al cuadrado y multiplicadas entre sí
df_aux['TV_sq'] = df_aux['TV'] ** 2
df_aux['Radio_sq'] = df_aux['Radio'] ** 2
df_aux['TV_Radio'] = df_aux['TV'] * df_aux['Radio']

# 16. Cálculo Manual de la prueba de White
df_aux_const = sm.add_constant(df_aux)
modelo_aux = sm.OLS(resid_sq, df_aux_const).fit()
r2_aux = modelo_aux.rsquared
white_manual = len(residuales) * r2_aux

# 17. Cálculo con Librería (statsmodels)
# het_white devuelve una tupla, el índice [0] es el estadístico LM (Lagrange Multiplier)
white_lib = het_white(residuales, X_train_final_const)[0]

# 18. Obtención del valor crítico (Nivel de confianza 90% -> alpha 0.10)
# Grados de libertad = 5 (porque usamos TV, Radio, TV_sq, Radio_sq, TV_Radio en la auxiliar)
chi2_critico_white = stats.chi2.ppf(0.90, df=5)

print("--- Supuesto 3: Homocedasticidad (Prueba de White) ---")
print(f"Estadístico White -> Manual: {white_manual:.4f} | Librería: {white_lib:.4f}")
print(f"Valor Crítico (Chi-cuadrada al 90% con 5 gl): {chi2_critico_white:.4f}")
```

```
--- Supuesto 3: Homocedasticidad (Prueba de White) ---
Estadístico White -> Manual: 11.3534 | Librería: 11.3534
Valor Crítico (Chi-cuadrada al 90% con 5 gl): 9.2364
```

D. Supuesto de Multicolinealidad (VIF)

Finalmente, se calculó el Factor de Inflación de Varianza (VIF) para descartar redundancia entre TV y Radio.

- **Interpretación:** Con valores de 1.0014 (menores a 5), se confirma que no existe multicolinealidad en el modelo.

```python
from statsmodels.stats.outliers_influence import variance_inflation_factor

# 19. Cálculo Manual de VIF
# VIF para TV (Regresión de TV sobre Radio)
modelo_vif_tv = sm.OLS(X_train_final['TV'], sm.add_constant(X_train_final[['Radio']])).fit()
vif_tv_manual = 1 / (1 - modelo_vif_tv.rsquared)

# VIF para Radio (Regresión de Radio sobre TV)
modelo_vif_radio = sm.OLS(X_train_final['Radio'], sm.add_constant(X_train_final[['TV']])).fit()
vif_radio_manual = 1 / (1 - modelo_vif_radio.rsquared)

# 20. Cálculo con Librería (statsmodels)
# variance_inflation_factor espera un array y el índice de la variable
# X_train_final_const tiene: [0] Constante, [1] TV, [2] Radio
vif_tv_lib = variance_inflation_factor(X_train_final_const.values, 1)
vif_radio_lib = variance_inflation_factor(X_train_final_const.values, 2)

print("--- Supuesto 4: Multicolinealidad (VIF) ---")
print(f"VIF TV    -> Manual: {vif_tv_manual:.4f} | Librería: {vif_tv_lib:.4f}")
print(f"VIF Radio -> Manual: {vif_radio_manual:.4f} | Librería: {vif_radio_lib:.4f}")
```

```
--- Supuesto 4: Multicolinealidad (VIF) ---
VIF TV    -> Manual: 1.0014 | Librería: 1.0014
VIF Radio -> Manual: 1.0014 | Librería: 1.0014
```

Conclusiones:

El modelo desarrollado es altamente efectivo para la predicción de ventas basándose en TV y Radio, explicando el 89.5% de la variabilidad. Aunque el modelo cumple con los supuestos de independencia (Durbin-Watson) y no redundancia (VIF), presenta retos en la normalidad y la homocedasticidad de sus errores, lo que debe considerarse al proyectar escenarios de inversión extrema.